

مقدمة:

يهدف هذا المشروع إلى تصميم وتطوير نظام إدارة مصرفية (banking system) يعتمد على مبادئ البرمجة غرضية التوجه، مع توظيف مجموعة من أنماط التصميم (Design Patterns) من أجل بناء نظام مرن، قابل للتوسع، وسهل الصيانة

تم التركيز في هذا المشروع على فصل المسؤوليات، تقليل الترابط بين المكونات، وتسهيل إضافة ميزات جديدة دون الحاجة إلى تعديل جذري في الكود البرمجي.

أهداف المشروع:

- تطبيق أنماط التصميم بشكل عملي داخل نظام واقع.
- تحسين قابلية التوسع والصيانة.
- التخزين والإشعارات وتحقيق الفصل بين المنطق البرمجي (Business Logic).
- بناء نظام منظم وقابل للاختبار.

نظرة عامة على بنية النظام:

يتكون النظام من عدة حزم كل حزمة مسؤولة عن جانب محدد من النظام مثل : -إدارة الحسابات (accounts) - التقارير (reports) - التوصيات (recommendation) - الإشعارات (notification) - المعاملات المالية (transaction) - التخزين المؤقت والمستودعات (repositories) - الأمان (security) .

أنماط التصميم المستخدمة (Design patterns):

○ Factory Pattern :

الوصف: نمط Factory يستخدم لإنشاء الكائنات دون تحديد الصنف (class) الفعلي الذي سيتم إنشاؤه.

التطبيق في المشروع: تم تطبيق هذا النمط في AccountFactory حيث يتم إنشاء أنواع مختلفة من الحسابات مثل:

- SavingsAccount
- CheckingAccount
- LoanAccount

InvestmentAccount •

AccountGroup •

بناءً على قيمة نصية (String) يتم تحديد نوع الحساب المطلوب إنشاؤه.

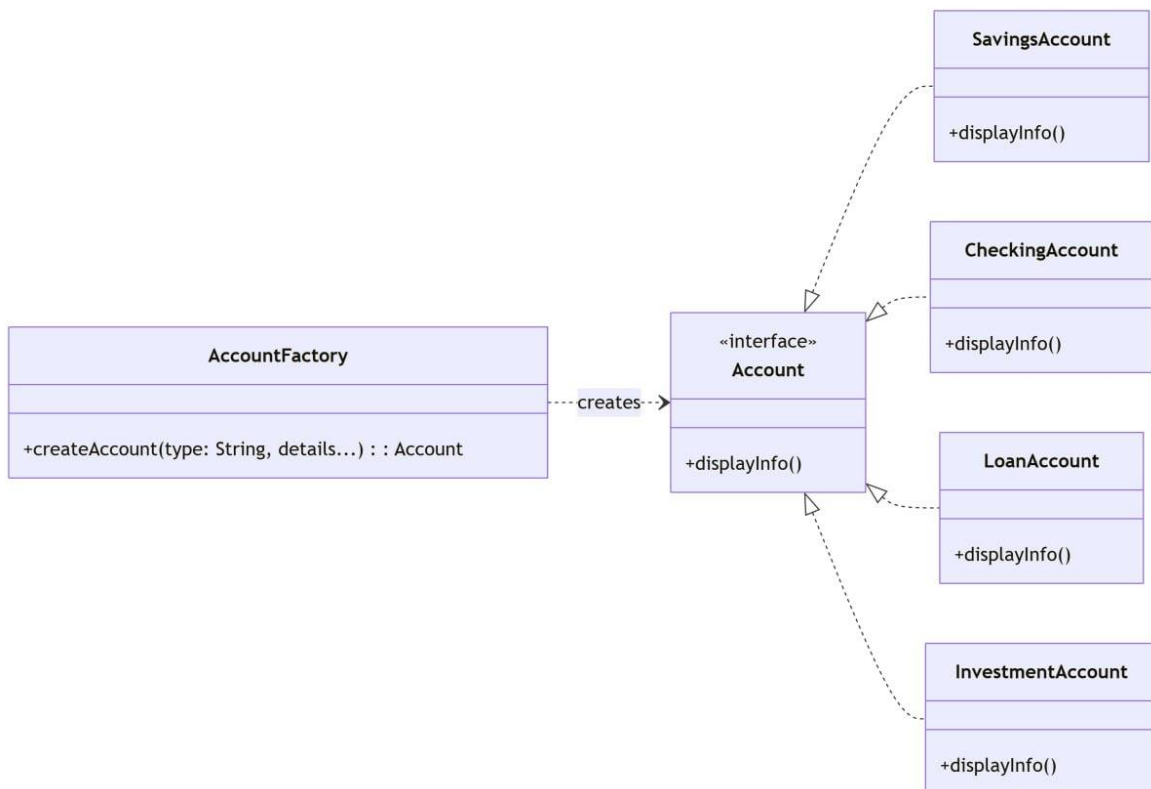
الفائدة:

- عزل منطق الإنشاء عن بقية النظام.
- تسهيل إضافة أنواع حسابات جديدة دون تعديل الكود المستخدم.

البدائل:

- استخدام Constructors مباشرة (غير مرن).
- (Abstract Factory) أكثر تعقيداً وغير مطلوب هنا.

□ مخطط ال UML لل Factory Patterns :



: Strategy Pattern 🚩

الوصف : يُستخدم Strategy لتغيير سلوك كائن ما أثناء وقت التشغيل دون تعديل الكود الأساسي.

التطبيق في المشروع : تم استخدامه في:

- حساب الفائدة InterestStrategy

- التوصيات RecommendationStrategy

حيث تم تعريف واجهات عامة وتطبيق استراتيجيات مختلفة مثل:

- SimpleInterestStrategy

- NoInterestStrategy

- LowBalanceRecommendation

- HighSpendingRecommendation

الفائدة:

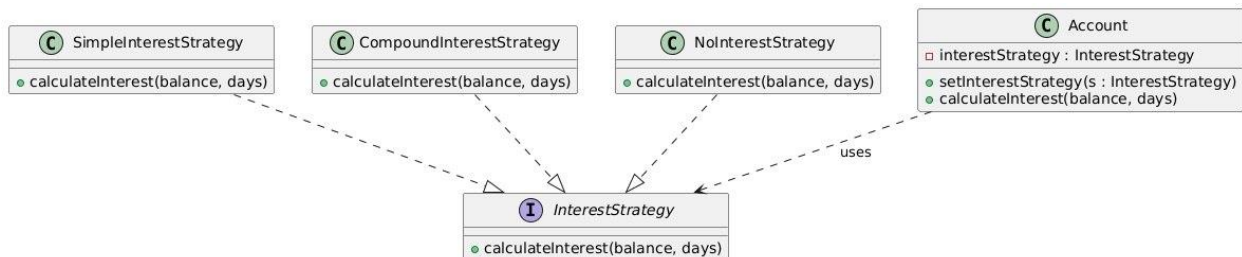
- إضافة استراتيجيات جديدة بسهولة.

- الالتزام بمبدأ Open/Closed Principle.

البدائل:

- استخدام شروط (if/else) غير مرن وصعب الصيانة.

UML – Strategy Pattern : 📌



:Observer Pattern 🚀

الوصف: يسمح هذا النمط بربط عدة مراقبين (Observers) بكائن واحد يتم إشعارهم عند حدوث تغيير.

التطبيق في المشروع: تم تطبيقه في نظام الإشعارات عبر:

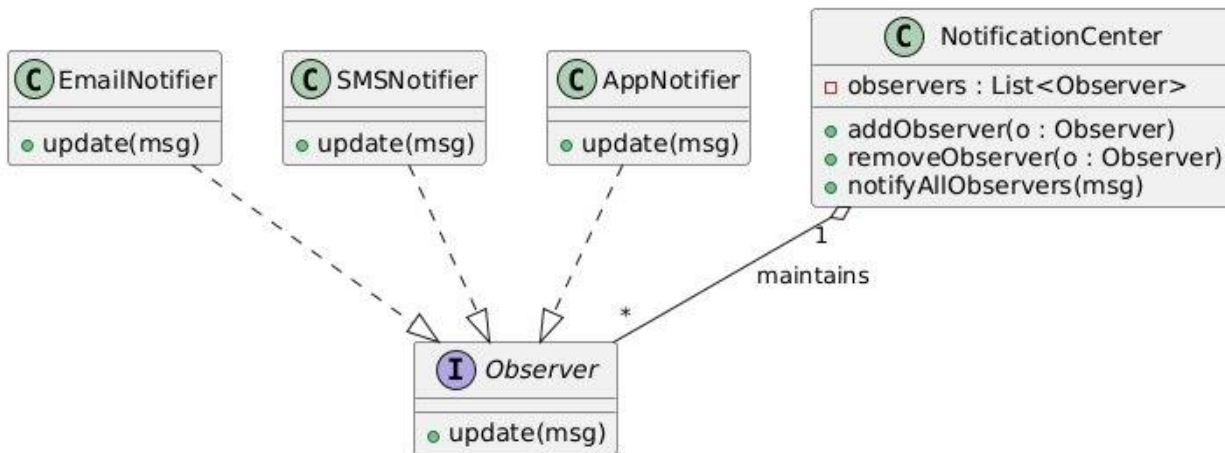
- NotificationCenter
- EmailNotifier
- SMSNotifier
- AppNotifier

عند حدوث حدث معين، يتم إرسال إشعار لجميع المشتركين.

الفائدة:

- فصل منطق الإشعارات عن منطق العمل الأساسي.
- دعم قنوات إشعار متعددة.

: UML – Observer Pattern 🚀



: State Pattern 🚀

الوصف: يسمح للكائن بتغيير سلوكه عند تغيير حالته الداخلية.

التطبيق في المشروع:

- AccountState
- ActiveState
- ClosedState
- FrozenState

كيف تُطبق: الحساب (Account) يحتوي على كائن يمثل حالته، ويتم تفويض السلوك (سحب، إيداع، إغلاق) إلى هذا الكائن.

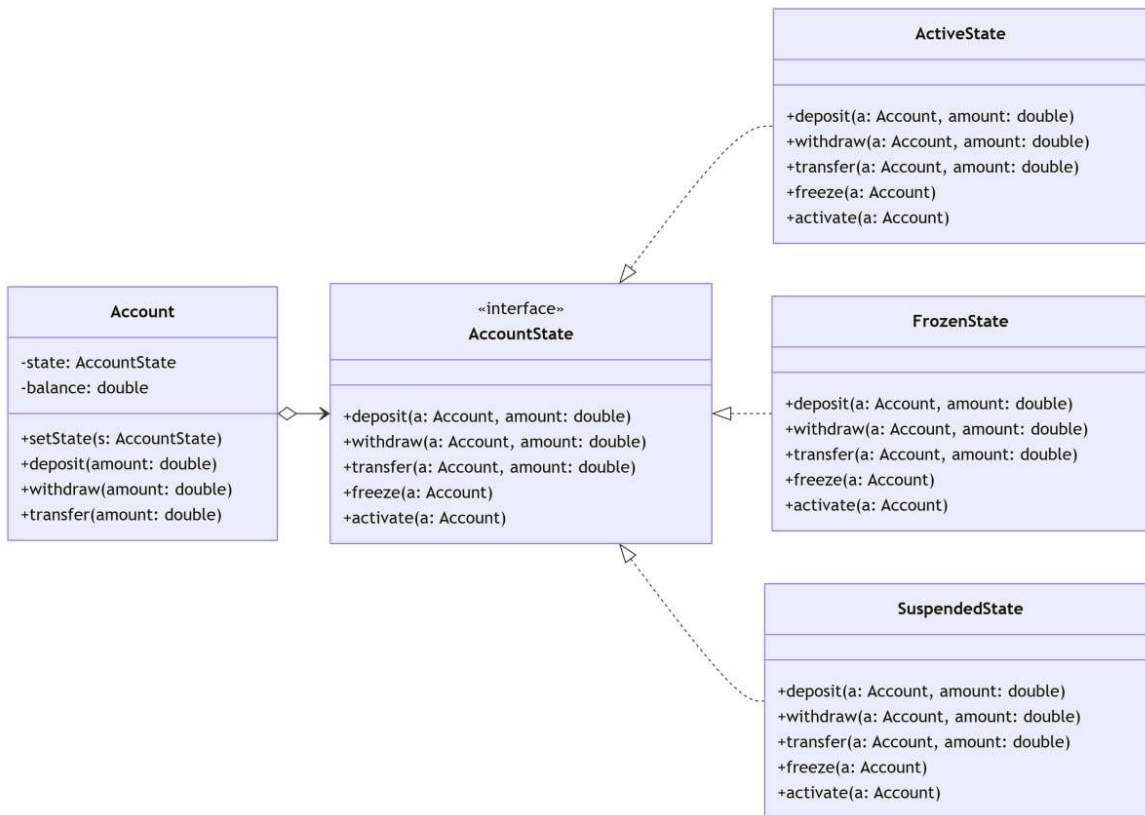
السبب:

- تجنب الشروط المتعددة.
- تسهيل إضافة حالات جديدة.

البديل:

- if / switch على حالة الحساب.

UML- state patterns ✎



:Composite Pattern

الوصف: التعامل مع الكائنات الفردية والمجموعات بشكل موحد.

التطبيق:

• Account

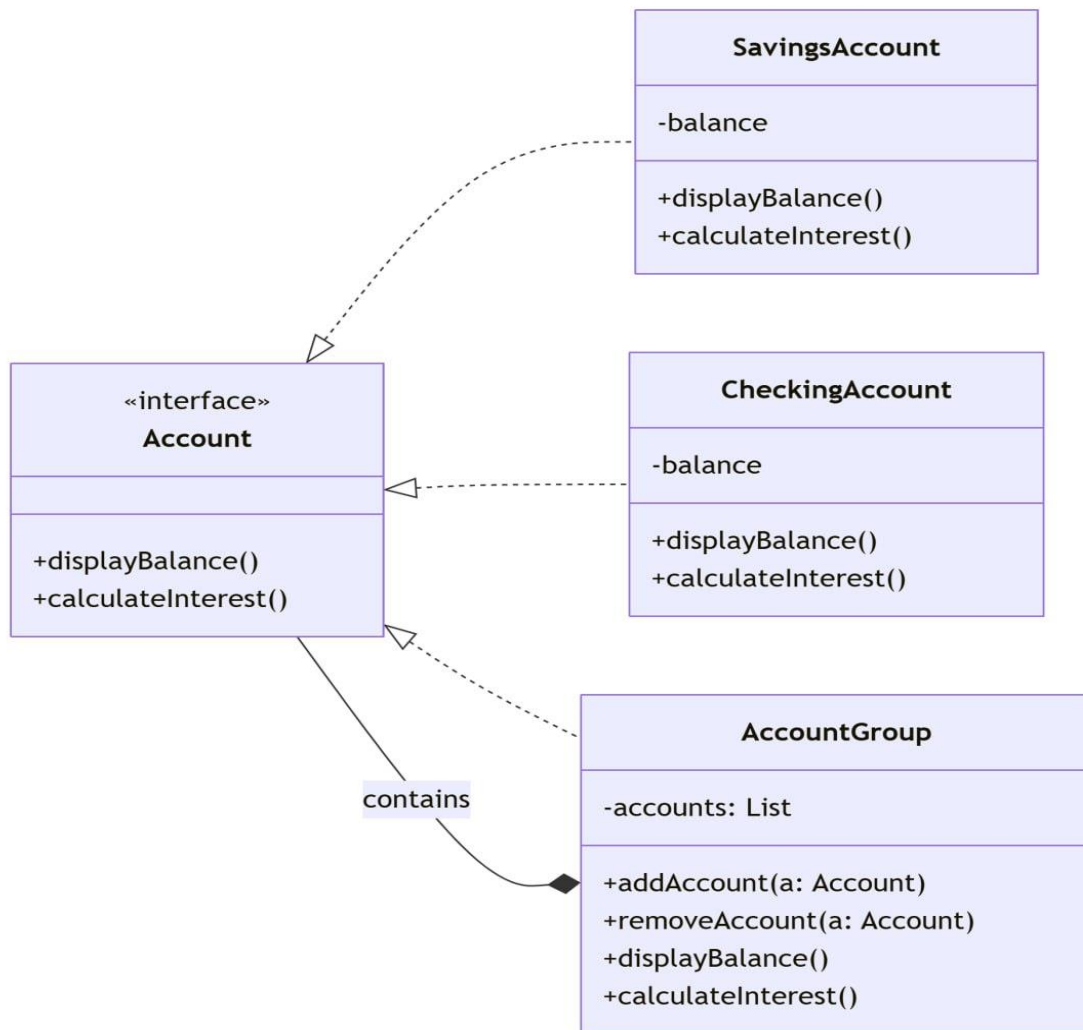
• AccountGroup

كيف تُطبق AccountGroup: يحتوي مجموعة من الحسابات ويُعامل كحساب واحد.

السبب:

• دعم الحسابات الجماعية.

 UML- composite patterns :



: Facade Pattern 🚀

الوصف: توفير واجهة مبسطة لنظام معقد.

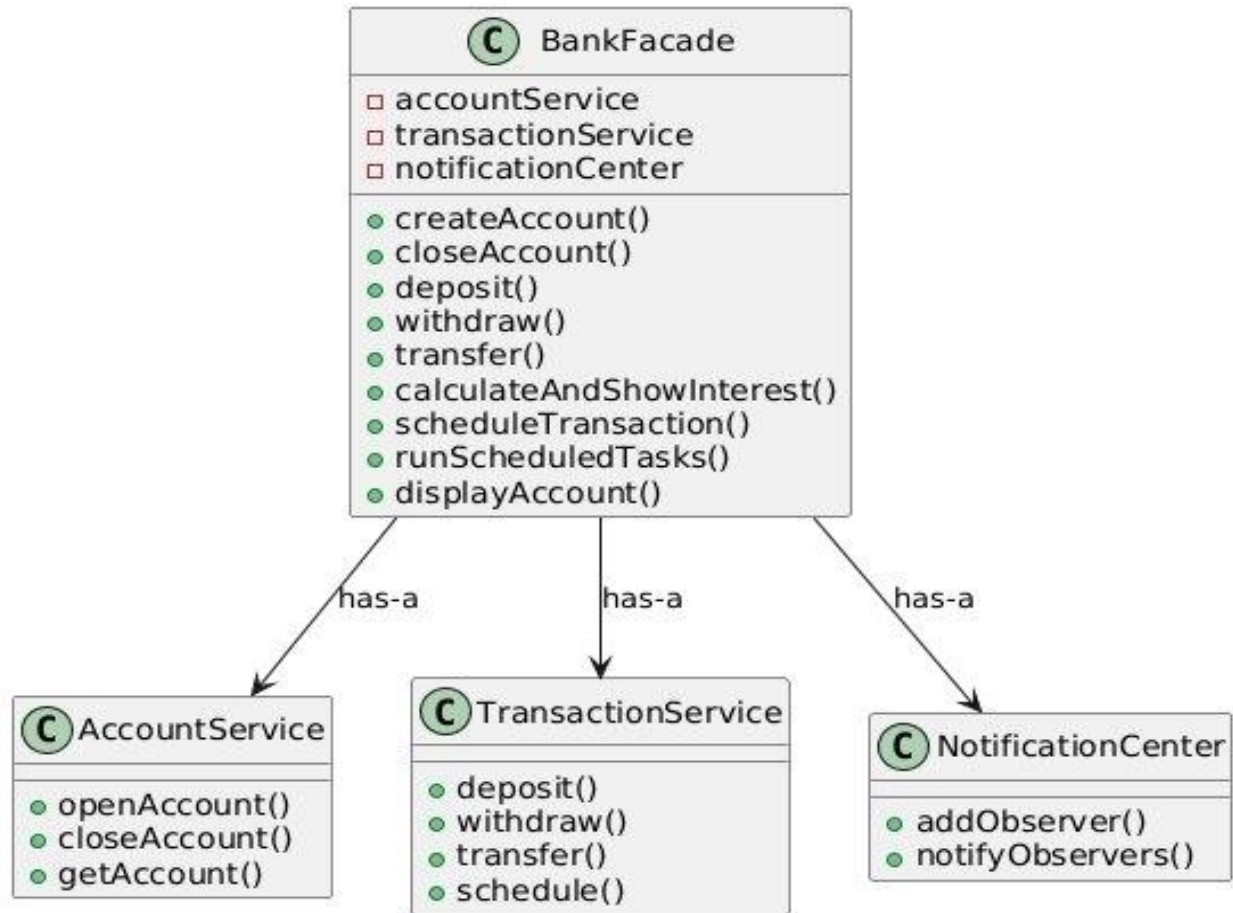
التطبيق:

- AccountService
- TransactionService
- ReportService

السبب:

- إخفاء تعقيد النظام عن المستخدم.

: UML-façade pattern 🗑



:Chain of Responsibility Pattern 🗑

الوصف: يسمح بتمرير الطلب عبر سلسلة من المعالجات حتى يتم التعامل معه.

التطبيق في المشروع: تم استخدامه في معالجة المعاملات المالية:

- `AutoApprovalHandler`
- `TellerApprovalHandler`
- `ManagerApprovalHandler`

حيث يتم اتخاذ قرار الموافقة حسب قيمة المعاملة.

: 🗑 UML – Chain of Responsibility Pattern

المتطلبات غير الوظيفية (Non-Functional Requirements):

➤ الصيانة (Maintainability) :

• Lombok :

الهدف: تقليل التكرار البرمجي وتحسين قابلية الصيانة.

كيف تم استخدامه:

- تم استخدام Lombok لتوليد:

- Constructors

- Getters / Setters

- Builders

- يظهر ذلك بوضوح في أصناف مثل:

- CheckingAccount

- SimpleInterestStrategy

- ScheduledTransaction

- NotificationCenter

الفائدة:

- تقليل حجم الكود

- سهولة التعديل مستقبلاً

- تقليل الأخطاء البشرية

البديل ولماذا لم يُستخدم:

- كتابة الكود يدوياً → يزيد التعقيد وصعوبة الصيانة

• Checkstyle :

الهدف: فرض أسلوب ترميز موحد وتحسين جودة الكود.

كيف يخدم المشروع:

- يضمن التزام جميع أعضاء الفريق بنفس Coding Standards

- يسهل قراءة الكود ومراجعته

- يدعم العمل الجماعي (5 طلاب)

ملاحظة: تم استخدامه كأداة ضبط جودة وليس كجزء من منطق التنفيذ.

➤ الأداء (Performance) :

• Cache (LRU Cache)

الهدف: تحسين سرعة الوصول للبيانات وتقليل الحمل على Repository.

كيف تم تطبيقه في الكود:

• AccountCache

• TransactionCache

• يعتمد على LRU Cache مع:

◦ Capacity

◦ Time-To-Live (TTL)

أماكن الاستخدام:

• AccountService.getAccount()

• TransactionService.getTransaction()

الفائدة:

• تقليل الوصول المتكرر للذاكرة أو التخزين

• تحسين زمن الاستجابة

البديل ولماذا لم يُستخدم:

• الوصول المباشر دائماً إلى Repository أبطأ وأقل كفاءة.

➤ الاختبار (Testability) :

• :JUnit 5

الهدف: اختبار الوحدات البرمجية. (Unit Testing)

كيف يخدم المشروع:

• اختبار الخدمات مثل:

◦ AccountService

◦ TransactionService

• التحقق من صحة منطق العمل بدون تشغيل النظام كاملاً.

• :Mockito

الهدف: عزل الوحدات أثناء الاختبار. (Mocking)

كيف يخدم المشروع:

- محاكاة:

- Repositories

- External Services

- اختبار منطق الخدمة بدون الاعتماد على بيانات حقيقية

الفائدة:

- اختبارات أسرع

- نتائج أدق

➤ الأمان (Security) :

- JWT (JSON Web Token)

الهدف: المصادقة (Authentication) وتحديد الصلاحيات. (Authorization)

كيف تم تطبيقه في الكود:

- JwtUtil:

- generateToken()

- validateToken()

- AuthenticatedUser

- AuthorizationManager.checkRole()

الوظيفة:

- التأكد من هوية المستخدم

- التحقق من الدور (ADMIN, MANAGER, TELLER, CUSTOMER)

الفائدة:

- فصل منطق الأمان عن منطق العمل

- دعم أنظمة موزعة مستقبلاً

الخلاصة:

تم في هذا المشروع بناء نظام مصرفي متكامل يعتمد على أنماط تصميم متعددة تم اختيارها بعناية لتناسب متطلبات النظام. أثبتت هذه الأنماط فعاليتها في تحسين جودة التصميم، تقليل التعقيد، وزيادة مرونة النظام وقابليته للتطوير.
